DATA 266 – Homework 3: Findings

Prompt Engineering (LangChain) & Self-Attention with Causal Masking from Scratch

Name: Rohil Utture | **SID4:** 2708 | **Seed:** 2708 | **Model used:** llama3.1 (8B), local via Ollama, temperature 0

Part 1 - Prompt Engineering with LangChain

Method. Each of the six techniques was implemented as a LangChain LCEL chain (`prompt | llm | StrOutputParser()`) and invoked programmatically - no chat-history screenshots. The model was **llama3.1 (8B) served locally through Ollama** at temperature 0 for reproducibility. Every technique was tested on **two** different tasks (math, logic, or text) as required.

Note: all outputs below are from the actual notebook run. Edit any wording you like - this is your editable copy; export to PDF for final submission.

Technique	Two example tasks	Actual result (llama3.1, temp 0)	What it demonstrates / finding
Zero-Shot	(A) 17 x 24 (B) sentiment of a bad review	(A) 408 [correct] (B) Negative [correct]	Direct recall handles easy single-step tasks; no scaffolding needed.
Few-Shot	(A) unit-price word problems (B) city/date extraction	(A) $54 [correct] - llama also showed its multiplication (B) city: Berlin, date: August 21 [correct]	Exemplars fix the output format; note llama volunteered reasoning on a pure few-shot prompt.
Chain-of-Thought	(A) 23 - 17 + 14 muffins (B) rose/flower syllogism	(A) 20 [correct, with steps] (B) No [correct, names the fallacy]	Demonstrated reasoning yields correct multi-step answers and shows the work.
Zero-Shot CoT	(A) bat & ball $1.10 (B) 100 days after Wednesday	(A) $0.05 [correct] (B) Friday [correct]	The trigger 'Let's think step by step' alone elicits correct reasoning; fixes the $0.10 trap.
Meta-Prompting	(A) 60 km in 45 min -> km/h (B) design a prompt, then answer	(A) 80 km/h [correct] - writes the method first (B) prompt + kid-friendly bullets	The model plans the method/prompt first, then executes it.
Tree of Thoughts	(A) pet logic puzzle (B) Game of 24 on 4,4,6,8	(A) Anna has the cat [correct] - full branch/critique/prune trace (B) FAILED - degenerated into a malformed brute-force search, never reached 24, and lost coherence	Structures a multi-branch search; but the 8B local model could not actually solve Game-of-24 - prompting structure does not equal capability.

Key findings

- **Model.** All results come from llama3.1 (8B) run locally via Ollama at temperature 0, so the run is reproducible and required no API key.
- **Reasoning scaffolds help exactly where the task is multi-step.** On the single-step tasks (17x24, sentiment) zero-shot already succeeds, so CoT would add cost with no accuracy gain. On the multi-step tasks (muffins, syllogism, bat & ball) the reasoning techniques are what make the answer correct and inspectable.
- **Zero-Shot CoT is the cheapest reliable fix for 'trap' questions.** A single trigger phrase drove llama to the correct $0.05 on the bat-and-ball problem, reproducing most of the benefit of hand-written CoT exemplars without authoring any.
- **Few-Shot is about form - but a capable instruct model blurs the line.** Its main effect was enforcing the output shape (the city/date schema). Interestingly, on Few-Shot Example A llama spontaneously wrote out its multiplication even though the exemplars showed only final answers - i.e. an 8B instruct model injects chain-of-thought into a few-shot prompt on its own.
- **Tree of Thoughts structured the search but did not guarantee a correct answer.** Example A (pet logic puzzle) worked cleanly: three 'experts' branched, critiqued each other's flawed steps, pruned, and converged on 'Anna

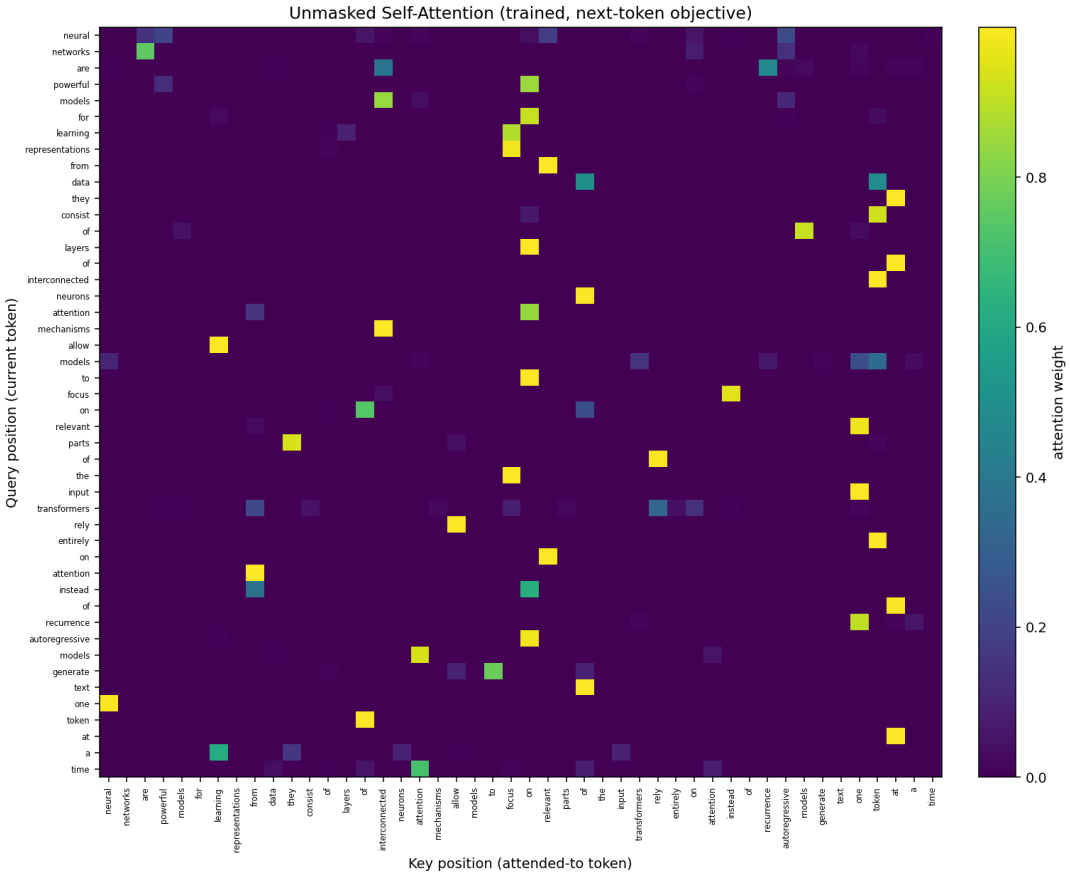
has the cat'. Example B (Game of 24 on 4,4,6,8) FAILED: llama degenerated into a huge malformed brute-force search, never reached 24, and broke character (it started asking the user what was going on). The correct solution is (6 - 4) x (8 + 4) = 24. This is the clearest finding of Part 1: the prompting scaffold shapes how the model searches, but on hard arithmetic a small local model can still fail - structure is not the same as capability.

- **Cost / latency ordering** (shortest -> longest output on these examples): Zero-Shot < Few-Shot < Zero-Shot CoT ~ CoT < Meta-Prompting < Tree of Thoughts. Tree of Thoughts was by far the slowest cell.

Part 2 - Self-Attention & Causal Masking from Scratch

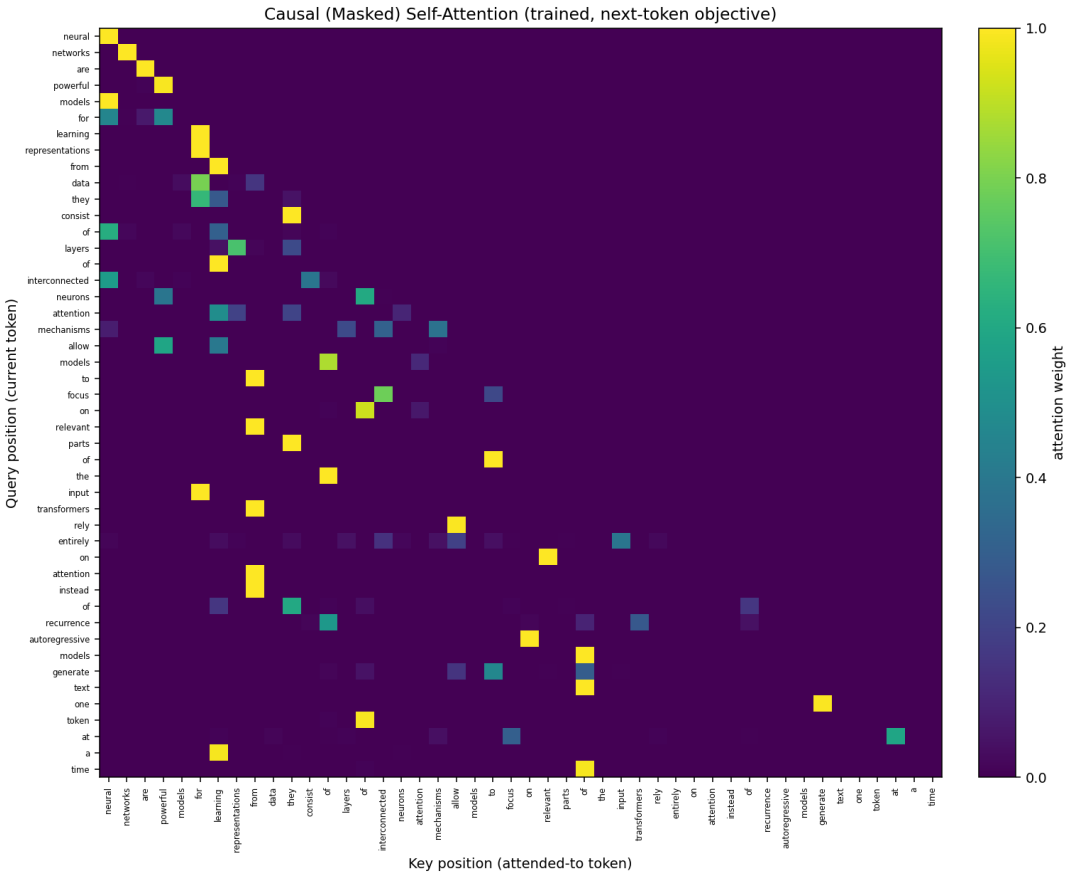
Setup. The required 5-sentence text was tokenized at the word level (lowercase, periods dropped) into T = 46 tokens over a vocabulary of 39 words. A trainable `nn.Embedding` (d_model = 64) plus a learned positional embedding feed a single-head scaled dot-product self-attention block built from scratch - three `nn.Linear` projections for Q, K, V, scores $QK^T/\sqrt{d}$, softmax, then weights x V. No `nn.MultiheadAttention`, `nn.Transformer`, or HuggingFace was used. The model is trained with a next-token cross-entropy objective (700 epochs, Adam, lr = 5e-3, seed = 2708). The unmasked and masked networks are the two trained models and share identical initialization, so the mask is the only difference.

Unmasked self-attention (Part 1 deliverable)



The learned weights are sharp and highly specific - clearly not the flat matrix of random initialization. Bright cells appear on *both* sides of the diagonal, i.e. a token may attend to positions that come **after** it. This bidirectional behaviour is exactly what 'unmasked' self-attention permits.

Causal (masked) self-attention (Part 2 deliverable)



A lower-triangular mask sets every future score to -infinity **before** the softmax. The result is strictly lower-triangular: the entire upper triangle is zero. The first token attends only to itself, and every later token spreads its attention across itself and earlier tokens only. This is the decoder self-attention used for autoregressive generation.

Sanity checks (from the executed notebook)

Check	Result	Meaning
max weight in upper triangle (masked)	0.0	no attention to future positions
each masked row sums to 1	True	valid probability distribution per query
non-zero future weights in row 0 (unmasked)	35	bidirectional attention confirmed
attention matrix shape	46 x 46	full-sequence self-attention

Discussion & conclusion

The two heatmaps make the core idea visible: masking changes *which* positions a query may look at, not the attention mechanism itself. Training loss reaches about 0 within the first hundred epochs because the dataset is a single short sequence that the network memorizes - expected for this from-scratch exercise, whose goal is to visualize *learned* attention structure rather than to generalize. One honest caveat for the unmasked model: because it is trained on a next-token objective without a mask, position t can see the answer token at t+1, so some of its bright off-diagonal cells reflect that leakage; the masked model removes this by construction, which is precisely why causal masking is required for autoregressive decoders.